

Functions

Content notes, classwork and homework with worked C++ solutions.

Lesson 1: Functions - Basics

Quick recap: Declaration and definition • Parameters and return values • Pass by value • Scope

Theory & Nuances

Declaration and definition

A function is a named block of code you can run whenever you need it, instead of copying the same lines again. The declaration (prototype) tells the compiler the name, return type and parameter types: `int factorial(int n)`; The definition supplies the body in curly braces. A function must be declared before it is called, so either define it above `main()` or put the prototype above `main()` and the full definition below.

Parameters and return values

Parameters are the inputs listed in the brackets; the values you pass when calling are called arguments. The return statement sends one value back to the caller and immediately ends the function. The return type must match what you send back - `bool` for yes/no answers, `int` for counts, `double` for decimals, and `void` when the function only does work (like printing) and returns nothing.

Pass by value

By default C++ copies each argument into the parameter. The function works on its own copy, so changing the parameter inside the function does NOT change the caller's variable. That is why `sumOfDigits(n)` can keep doing `n /= 10` without destroying the number back in `main()`.

Scope

A variable lives only inside the curly braces where it was declared. Variables declared inside a function (and its parameters) are local: `main()` cannot see them, and two different functions can each have their own variable named `n` without any clash. A global variable, declared outside every function, is visible everywhere but should be used sparingly.

Key Points / Common Mistakes

- Calling a function before it is declared is a compile error - define it above `main()` or add a prototype.
- A non-void function must return a value on every path, including the 'else' path.
- Changing a pass-by-value parameter never affects the caller's variable.
- `isPrime`: 0 and 1 are not prime, and you only need to test divisors up to `sqrt(n)` (`i * i <= n`).

Classwork Problems

1. isPrime(n)

```
#include <iostream>
using namespace std;

bool isPrime(int n) {
    if (n < 2) return false;
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) return false;
    }
    return true;
}

int main() {
    cout << "Primes up to 20: ";
    for (int x = 1; x <= 20; x++) {
        if (isPrime(x)) cout << x << " ";
    }
    cout << endl;
    return 0;
}
```

Output

```
Primes up to 20: 2 3 5 7 11 13 17 19
```

2. factorial(n)

```
#include <iostream>
using namespace std;

long long factorial(int n);    // declaration (prototype)

int main() {
    for (int i = 0; i <= 5; i++) {
        cout << i << "! = " << factorial(i) << endl;
    }
    return 0;
}

long long factorial(int n) {    // definition
    long long result = 1;
    for (int i = 2; i <= n; i++) {
        result *= i;
    }
    return result;
}
```

Output

```
0! = 1
1! = 1
2! = 2
3! = 6
4! = 24
5! = 120
```

3. max(a, b) and pass by value

```
#include <iostream>
using namespace std;

int getMax(int a, int b) {
    if (a > b) return a;
    return b;
}

void tryToChange(int x) {
    x = 100;    // changes only the local copy
    cout << "Inside tryToChange, x = " << x << endl;
}

int main() {
    int p = 17, q = 42;
    cout << "Max of " << p << " and " << q << " is " << getMax(p, q) << endl;

    tryToChange(p);
    cout << "After tryToChange(p), p is still " << p << endl;
    return 0;
}
```

Output

```
Max of 17 and 42 is 42
Inside tryToChange, x = 100
After tryToChange(p), p is still 17
```

Homework Problems

1. sumOfDigits(n)

```
#include <iostream>
using namespace std;

int sumOfDigits(int n) {
    int sum = 0;
    while (n > 0) {
        sum += n % 10;
        n /= 10;
    }
    return sum;
}

int main() {
    int num = 98765;
    cout << "Sum of digits of " << num << " = " << sumOfDigits(num) << endl;
    cout << "num in main is unchanged: " << num << endl;
    return 0;
}
```

Output

```
Sum of digits of 98765 = 35
num in main is unchanged: 98765
```

2. isPalindromeNumber(n)

```
#include <iostream>
using namespace std;

bool isPalindromeNumber(int n) {
    int original = n, reversed = 0;
    while (n > 0) {
        reversed = reversed * 10 + n % 10;
        n /= 10;
    }
    return original == reversed;
}

int main() {
    int a = 12321, b = 1234;
    cout << a << (isPalindromeNumber(a) ? " is" : " is not") << " a palindrome" << endl;
    cout << b << (isPalindromeNumber(b) ? " is" : " is not") << " a palindrome" << endl;
    return 0;
}
```

Output

```
12321 is a palindrome
1234 is not a palindrome
```

Lesson 2: Multi-parameter Functions

Quick recap: Several parameters • Calling functions from functions • Returning results • Default arguments

Theory & Nuances

Several parameters

A function can take as many parameters as it needs, separated by commas, each with its own type: double areaOfRectangle(double length, double width). Arguments are matched to parameters by position, so the order you pass them in matters - power(2, 5) and power(5, 2) give very different answers.

Calling functions from functions

Any function can call another function, not just main(). This lets you build big ideas from small, tested pieces: nCr calls factorial three times, and maxOfThree simply calls maxOfTwo twice. Each call gets its own fresh set of local variables and returns to exactly where it was called from.

Returning results

return hands back one value. That value can be used anywhere an expression can: stored in a variable, printed directly, passed into another function, or used in a condition like if (isLeapYear(y)). A bool-returning function can return the condition itself - return a > b; - instead of writing if/else that returns true/false.

Default arguments

A parameter can be given a default value: int power(int base, int exp = 2). If the caller leaves that argument out, the default is used, so power(7) means power(7, 2). Defaults must be the rightmost parameters, and they are written once - in the prototype if there is one, otherwise in the definition.

Key Points / Common Mistakes

- Arguments are matched by position, not by name.
- Default arguments must come last: int f(int a = 1, int b) is an error.
- Factorials overflow int quickly (13! is too big) - use long long.
- Circle area needs double; 22/7 is integer division and gives 3.

Classwork Problems

1. nCr using a factorial helper

```
#include <iostream>
using namespace std;

long long factorial(int n) {
    long long result = 1;
    for (int i = 2; i <= n; i++) {
        result *= i;
    }
    return result;
}

long long nCr(int n, int r) {
    return factorial(n) / (factorial(r) * factorial(n - r));
}

int main() {
    int n = 6, r = 2;
    cout << n << "C" << r << " = " << nCr(n, r) << endl;
    cout << "5C3 = " << nCr(5, 3) << endl;
    return 0;
}
```

Output

```
6C2 = 15
5C3 = 10
```

2. power(base, exp) with a default argument

```
#include <iostream>
using namespace std;

long long power(int base, int exp = 2) {
    long long result = 1;
    for (int i = 1; i <= exp; i++) {
        result *= base;
    }
    return result;
}

int main() {
    cout << "2^10 = " << power(2, 10) << endl;
    cout << "3^4 = " << power(3, 4) << endl;
    cout << "7^2 = " << power(7) << " (exp uses its default 2)" << endl;
    return 0;
}
```

Output

```
2^10 = 1024
3^4 = 81
7^2 = 49 (exp uses its default 2)
```

3. Area functions for shapes

```
#include <iostream>
using namespace std;

double areaOfCircle(double radius) {
    return 3.14159 * radius * radius;
}

double areaOfRectangle(double length, double width) {
    return length * width;
}

double areaOfTriangle(double base, double height) {
    return 0.5 * base * height;
}

int main() {
    cout << "Circle (r = 5): " << areaOfCircle(5) << endl;
    cout << "Rectangle (4 x 6): " << areaOfRectangle(4, 6) << endl;
    cout << "Triangle (b = 8, h = 3): " << areaOfTriangle(8, 3) << endl;
    return 0;
}
```

Output

```
Circle (r = 5): 78.5397
Rectangle (4 x 6): 24
Triangle (b = 8, h = 3): 12
```

Homework Problems

1. min/max of three via a function

```
#include <iostream>
using namespace std;

int maxOfTwo(int a, int b) {
    return (a > b) ? a : b;
}

int minOfTwo(int a, int b) {
    return (a < b) ? a : b;
}

int maxOfThree(int a, int b, int c) {
    return maxOfTwo(maxOfTwo(a, b), c);
}

int minOfThree(int a, int b, int c) {
    return minOfTwo(minOfTwo(a, b), c);
}

int main() {
    int x = 23, y = 71, z = 48;
    cout << "Numbers: " << x << ", " << y << ", " << z << endl;
    cout << "Max = " << maxOfThree(x, y, z) << endl;
    cout << "Min = " << minOfThree(x, y, z) << endl;
    return 0;
}
```

Output

```
Numbers: 23, 71, 48
Max = 71
Min = 23
```

2. isLeapYear(y) returning bool

```
#include <iostream>
using namespace std;

bool isLeapYear(int y) {
    return (y % 4 == 0 && y % 100 != 0) || (y % 400 == 0);
}

int main() {
    int years[4] = {1900, 2000, 2024, 2026};
    for (int i = 0; i < 4; i++) {
        if (isLeapYear(years[i])) cout << years[i] << " is a leap year" << endl;
        else cout << years[i] << " is not a leap year" << endl;
    }
    return 0;
}
```

Output

```
1900 is not a leap year
2000 is a leap year
2024 is a leap year
2026 is not a leap year
```

Lesson 3: Digit Problems

Quick recap: Reverse a number • Sum and count of digits • Palindrome number • Armstrong number

Theory & Nuances

Reverse a number

Every digit problem uses the same two tools: $n \% 10$ gives the last digit, and $n /= 10$ throws the last digit away. To reverse, peel off the last digit and append it to the answer with $\text{reversed} = \text{reversed} * 10 + \text{digit}$, repeating until n becomes 0. Trailing zeros disappear: the reverse of 1200 is 21.

Sum and count of digits

Sum of digits adds each peeled digit to an accumulator; count of digits just adds 1 per pass. Both loops run once per digit, so they take $O(\text{number of digits}) = O(\log_{10} N)$ time. Watch the special case $n = 0$: it has one digit, but a plain while ($n > 0$) loop counts zero - handle it separately.

Palindrome number

A number is a palindrome if it reads the same both ways, like 121 or 4554. Save a copy of the original, reverse the number, and compare. You need the copy because the reversing loop destroys n as it goes.

Armstrong number

An Armstrong number equals the sum of its digits each raised to the power of the number of digits. 153 has 3 digits and $1^3 + 5^3 + 3^3 = 153$, so it is Armstrong; $9474 = 9^4 + 4^4 + 7^4 + 4^4$ also is. This combines two earlier functions: countDigits to find the power, then a sum loop using that power.

Key Points / Common Mistakes

- Always work on a copy of n when you need the original afterwards.
- countDigits(0) should be 1, not 0 - test the zero case.
- Use integer power (a loop) for Armstrong; pow() returns double and can round badly.
- Digit loops run once per digit, so they are $O(\log N)$, not $O(N)$.

Classwork Problems

1. Reverse a number

```
#include <iostream>
using namespace std;

int reverseNumber(int n) {
    int reversed = 0;
    while (n > 0) {
        int digit = n % 10;
        reversed = reversed * 10 + digit;
        n /= 10;
    }
    return reversed;
}

int main() {
    cout << "Reverse of 12345 = " << reverseNumber(12345) << endl;
    cout << "Reverse of 1200 = " << reverseNumber(1200) << endl;
    return 0;
}
```

Output

```
Reverse of 12345 = 54321
Reverse of 1200 = 21
```

2. Sum of digits

```
#include <iostream>
using namespace std;

int sumOfDigits(int n) {
    int sum = 0;
    while (n > 0) {
        sum += n % 10;
        n /= 10;
    }
    return sum;
}

int main() {
    int nums[3] = {472, 9999, 1005};
    for (int i = 0; i < 3; i++) {
        cout << "Sum of digits of " << nums[i] << " = " << sumOfDigits(nums[i]) << endl;
    }
    return 0;
}
```

Output

```
Sum of digits of 472 = 13
Sum of digits of 9999 = 36
Sum of digits of 1005 = 6
```

3. Count digits

```
#include <iostream>
using namespace std;

int countDigits(int n) {
    if (n == 0) return 1;
    int count = 0;
    while (n > 0) {
        count++;
        n /= 10;
    }
    return count;
}

int main() {
    cout << "Digits in 0 = " << countDigits(0) << endl;
    cout << "Digits in 7 = " << countDigits(7) << endl;
    cout << "Digits in 90210 = " << countDigits(90210) << endl;
    return 0;
}
```

Output

```
Digits in 0 = 1
Digits in 7 = 1
Digits in 90210 = 5
```

Homework Problems

1. Palindrome number

```
#include <iostream>
using namespace std;

int reverseNumber(int n) {
    int reversed = 0;
    while (n > 0) {
        reversed = reversed * 10 + n % 10;
        n /= 10;
    }
    return reversed;
}

bool isPalindrome(int n) {
    return n == reverseNumber(n);
}

int main() {
    int nums[4] = {121, 4554, 123, 7};
    for (int i = 0; i < 4; i++) {
        cout << nums[i] << " -> " << (isPalindrome(nums[i]) ? "Palindrome" : "Not palindrome") << endl;
    }
    return 0;
}
```

Output

```
121 -> Palindrome
4554 -> Palindrome
123 -> Not palindrome
7 -> Palindrome
```

2. Armstrong number; largest digit

```
#include <iostream>
using namespace std;

int countDigits(int n) {
    if (n == 0) return 1;
    int count = 0;
    while (n > 0) {
        count++;
        n /= 10;
    }
    return count;
}

int power(int base, int exp) {
    int result = 1;
    for (int i = 1; i <= exp; i++) result *= base;
    return result;
}

bool isArmstrong(int n) {
    int digits = countDigits(n);
    int sum = 0, temp = n;
    while (temp > 0) {
        sum += power(temp % 10, digits);
        temp /= 10;
    }
    return sum == n;
}

int largestDigit(int n) {
    int largest = 0;
    while (n > 0) {
        if (n % 10 > largest) largest = n % 10;
        n /= 10;
    }
    return largest;
}

int main() {
    int nums[3] = {153, 9474, 123};
    for (int i = 0; i < 3; i++) {
        cout << nums[i] << (isArmstrong(nums[i]) ? " is" : " is not") << " an Armstrong number" << endl;
    }
    cout << "Largest digit of 58372 = " << largestDigit(58372) << endl;
    return 0;
}
```

Output

```
153 is an Armstrong number
9474 is an Armstrong number
123 is not an Armstrong number
Largest digit of 58372 = 8
```